학습 노트 인덱스

프로젝트: Dev 2 정적 분석 + 보안 스캔 에이전트
기준: 초보자 눈높이 (비유 -> 개념 -> 이유 -> 코드 순서)

Task 1~2 — 분석 노드 구현 (run_tools)

핵심: "외부 프로그램을 파이썬에서 어떻게 실행하고, 결과를 어떻게 병렬로 처리하나?"

영역	개념	파일	난이도
Python	subprocess — 외부 프로그램 실행	python/subprocess.md	[*]
보안	정적 분석 도구 (Pylint/Bandit/Semgrep/radon/pip-audit)	security/static_analysis.md	[][]
비동기	asyncio.gather — 병렬 비동기 실행	python/asyncio_gather.md	[][]
AI Native	다국어 라우터 (Python/JS/Java 파일별 도구 선택)	ai/multilang_router.md	[][]
AI Native	subprocess + async 조합	ai/subprocess_async.md	[][]
AI Native	병렬 에이전트 실행 패턴	ai/asyncio_parallel.md	[][]

자주 헷갈리는 것:
- Pylint vs Bandit 차이? -> Pylint = 코드 스타일, Bandit = 보안 취약점
- subprocess와 asyncio를 같이 쓰는 이유? -> 외부 프로그램 실행(subprocess) + 동시 실행(asyncio)

Task 3 — vector_search + llm_judge 노드

핵심: "KMS에서 관련 문서를 찾고, AI가 이슈를 판단한다"

영역	개념	파일	난이도
벡터	Vector DB / 임베딩 (텍스트 -> 숫자 변환)	vector/vectordb.md	[][][*]
AI Native	pgVector RAG (유사도 검색)	ai/pgvector_rag.md	[][][*]
AI Native	프롬프트 엔지니어링 (Few-shot + CoT)	ai/prompt_engineering.md	[][]
AI Native	LLM Judge 패턴 (Critical/Major/Minor 판정)	ai/llm_judge_patterns.md	[][][*]
에이전트	LangChain 체인	agent/langchain.md	[][]

자주 헷갈리는 것:
• 임베딩이 뭐야? -> 텍스트를 숫자 배열로 바꿔서 "비슷한 내용" 수학으로 계산
• CoT(Chain of Thought)가 뭐야? -> "단계별로 생각해"라고 AI에게 지시하는 기법

Task 4 — store 노드 (DB 저장)

핵심: "분석 결과를 PostgreSQL에 저장하고, 중복이면 덮어쓴다"

영역	개념	파일	난이도
DB	PostgreSQL 인덱스 종류 (B-tree, GIN, HNSW)	database/pg_indexes.md	[][][*]
DB	UPSERT (INSERT + 중복이면 UPDATE)	database/upsert.md	[*]
AI Native	store 노드 패턴	ai/store_node_patterns.md	[][]
AI Native	LangSmith 평가 / 추적	ai/langsmith_evaluation.md	[][]

자주 헷갈리는 것:
• [ON CONFLICT DO UPDATE]가 뭐야? -> 같은 key가 이미 있으면 UPDATE 실행 (= UPSERT)
• [await s.commit()] 왜 필요해? -> commit 없으면 DB에 실제로 저장 안 됨

Task 5 — LangGraph StateGraph 조립 (graph.py)

핵심: "5개 노드를 연결해서 파이프라인을 만든다"

영역	개념	파일	난이도
에이전트	MAS (멀티 에이전트 시스템)	agent/mas.md	[][]
에이전트	LangGraph StateGraph	agent/langgraph.md	[][][*]
AI Native	LangGraph 5단계 파이프라인	ai/langgraph_stategraph_pipeline.md	[][][*]

파이프라인 흐름:

```
run_tools -> vector_search -> llm_judge -> [escalate?] -> store
```

자주 헷갈리는 것:
• [state]가 뭐야? -> 모든 노드가 공유하는 딕셔너리 (데이터 저장소)
• [add_edge] vs [add_conditional_edges]? -> add_edge=항상 이동, conditional=조건부 분기

Task 6 — FastAPI 엔드포인트 (POST /scan + Insights API + PATCH dismiss)

핵심: "LangGraph 파이프라인을 HTTP API로 외부에 노출한다"

영역	개념	파일	난이도

API	FastAPI APIRouter / include_router (라우터 분리)	ai/fastapi_endpoints.md	[*]
API	HTTP 상태코드 202 Accepted vs 200 OK	ai/fastapi_endpoints.md	[*]
API	FastAPI BackgroundTasks (응답 후 비동기 실행)	ai/fastapi_endpoints.md	[][]
API	HTTP 동사 GET/POST/PATCH와 멱등성	ai/fastapi_endpoints.md	[][]
DB	SQLAlchemy AsyncSession + 파라미터 바인딩	ai/fastapi_endpoints.md	[][]
Python	Lazy import (함수 내부 import)	ai/fastapi_endpoints.md	[][]
Python	db.py Lazy 초기화 (싱글톤 엔진)	ai/fastapi_endpoints.md	[][]
테스트	Mock으로 async with 흉내내기 (MagicMock/AsyncMock)	ai/fastapi_endpoints.md	[][][*]
테스트	patch where it's used vs defined	ai/fastapi_endpoints.md	[][][*]
테스트	httpx ASGITransport (in-process API 테스트)	ai/fastapi_endpoints.md	[][]

Task 6 — FastAPI 엔드포인트 (POST /scan + Insights API + PATCH dismiss)

한 노트([ai/fastapi_endpoints.md])에 10개 개념을 비유->개념->이유->코드 순으로 정리.

영역	개념	파일	난이도
API	FastAPI APIRouter / include_router (라우터 분리)	ai/fastapi_endpoints.md	[*]
API	HTTP 상태코드 202 Accepted vs 200 OK	ai/fastapi_endpoints.md	[*]
API	FastAPI BackgroundTasks (응답 후 비동기 실행)	ai/fastapi_endpoints.md	[][]
API	HTTP 동사 GET/POST/PATCH와 멱등성	ai/fastapi_endpoints.md	[][]
DB	SQLAlchemy AsyncSession + 파라미터 바인딩	ai/fastapi_endpoints.md	[][]
Python	Lazy import (함수 내부		

	import)	ai/fastapi_endpoints.md	[][]
Python	db.py Lazy 초기화 (싱글톤 엔진)	ai/fastapi_endpoints.md	[][]
테스트	Mock으로 async with 흉내내기 (MagicMock/AsyncMock)	ai/fastapi_endpoints.md	[][][*]
테스트	patch where it's used vs defined	ai/fastapi_endpoints.md	[][][*]
테스트	httpx ASGITransport (in-process API 테스트)	ai/fastapi_endpoints.md	[][]

학습 목표 (면접·포트폴리오용)

- FastAPI 라우터 구조를 직접 설명할 수 있다
- 202 vs 200 차이를 면접에서 설명할 수 있다
- BackgroundTasks를 비유로 설명할 수 있다
- Mock으로 DB 없이 테스트하는 이유와 방법을 설명할 수 있다
- Lazy import를 왜 쓰는지 설명할 수 있다

복습 순서 추천

```
1일차: MAS -> LangGraph -> LangChain
2일차: asyncio.gather -> subprocess
3일차: Vector DB -> PostgreSQL 인덱스
4일차: Airflow -> n8n -> UPSERT
5일차: 정적 분석 도구 -> TDD
6일차: FastAPI 엔드포인트 (APIRouter -> 202/BackgroundTasks -> HTTP 동사 -> AsyncSession -> Lazy import -> Mock 테스트)
```